



Entregable Semana 7

Ajxup Yax Elisa Yanira - 1553222

Castro Ordoñez Karol Mariánn - 1545522

Menchú Calderón Wilder Lionel - 1518222

Marroquin Escobar Juan Diego - 1689821

Hernandez De León Anderson Omar Noé - 1512321

Quemé Cárcamo Jesús Marco Antonio - 1592920

Bustamante Orozco Guillermo Enrique - 1584515

Tzunún Gutiérrez Estuardo Alexander - 1605922

Planificación

Plan de Iteraciones

El desarrollo del proyecto se ejecutará mediante un plan estructurado de 5 Sprints, con una duración de una semana por iteración. Se proyecta la finalización de las tareas de desarrollo para aproximadamente el 20 de septiembre.

Debido a la dinámica del equipo, la metodología ágil se ha adaptado a la realidad del entorno académico: no se llevarán a cabo Daily Scrums tradicionales. En su lugar, el equipo mantendrá una sincronización constante a través de un grupo de WhatsApp para comunicación asíncrona, y se aprovecharán los dos períodos de clase a la semana como reuniones oficiales de control y seguimiento para resolver dudas y desbloquear tareas.

Cronograma Detallado

Se mantiene un cronograma detallado para todos los entregables a lo largo del curso.

GANTT

Además, para completar el sistema, el cronograma contempla la distribución de aproximadamente 250 issues a lo largo de las 5 semanas, con un promedio de asignación de 45 a 55 tareas semanales para todo el equipo.

La asignación de las tareas obedece a una cadena lógica de dependencias para asegurar un avance fluido, estructurada de la siguiente manera:

- Sprints Iniciales (Fundación): El esfuerzo arranca enfocándose exclusivamente en la infraestructura. Se prioriza el desarrollo del backend, la base de datos (DB) y la configuración de APIs, ya que son los cimientos sobre los que operará el resto del sistema.
- Sprints Intermedios (Core y Procesos): Una vez que la infraestructura está lista, el equipo avanza en paralelo hacia los módulos centrales (Gestión de Pacientes e Inventario). Al completarlos, se pasa a los flujos dependientes que consumen esos datos (Jornadas y Donaciones).

- Sprints Finales (Pulido y Pruebas): Las últimas iteraciones se concentran en el frontend, la generación de reportes, los roles de usuario y, finalmente, las issues exclusivas de pruebas integrales para garantizar la calidad antes de la entrega.

Semana 1:

Desarrollador	Issues Asignados
Karol	- [CHORE] Migracion: extensiones, tipos enumerados y funcion de timestamps - [CHORE] Migracion: tabla de perfiles de usuario y especialidades - [CHORE] Migracion: catalogo de permisos y su asignacion por rol y por usuario - [CHORE] Migracion: funciones auxiliares de autorizacion para RLS - [CHORE] Migracion: departamentos, municipios y comunidades - [CHORE] Migracion: tablas de pacientes y expedientes - [CHORE] Migracion: catalogo de condiciones cronicas y su asociacion al paciente
Wilder	- [CHORE] Leer y validar las variables de entorno en shared - [CHORE] Inicializar el cliente de Supabase en shared - [CHORE] Definir el adaptador de almacenamiento de sesion por plataforma - [CHORE] Normalizar los errores de Supabase en shared - [CHORE] Utilidades de fecha y formato en shared - [FEATURE] Validaciones de credenciales y datos de usuario en shared - [CHORE] Habilitar RLS en todas las tablas con denegacion por defecto
Jesus	- [CHORE] Definir la paleta de colores en ui-tokens - [CHORE] Definir escala de espaciado y tipografia en ui-tokens - [CHORE] Configurar el router de la aplicacion web

Desarrollador	Issues Asignados
	- [CHORE] Definir textos y etiquetas comunes en ui-tokens - [FEATURE] Pantalla de inicio de sesion en web - [CHORE] Integrar los tokens de diseno con react-bootstrap en web
Anderson	- [CHORE] Configurar la navegacion de la aplicacion movil - [CHORE] Layout base y componentes comunes de la aplicacion movil - [CHORE] Persistir la sesion movil con AsyncStorage - [FEATURE] Pantalla de inicio de sesion en la app movil - [CHORE] Configurar icono, splash y metadatos de la app movil - [RF] Cierre de sesion y cambio de usuario en la app movil
Estuardo	- [CHORE] Migracion: tabla de proyectos sociales - [CHORE] Migracion: jornadas, personal asignado e historial de estados - [CHORE] Migracion: principios activos y catalogo de medicamentos - [CHORE] Migracion: proveedores y bodegas - [CHORE] Migracion: indices de busqueda de pacientes por nombre sin acentos - [CHORE] Migracion: atenciones y triaje con signos vitales
Juan Diego	- [CHORE] Definir los tipos base del dominio en shared - [CHORE] Migracion: tabla de alertas de caducidad - [RF] Hook de sesion compartido con restauracion y refresco automatico - [CHORE] Migracion: vista de reporte de impacto - [CHORE] Migracion: registro de auditoria y borrado lógico - [CHORE] Migracion: donantes, donaciones y detalle de la donación - [CHORE] Migracion: lotes y existencias por bodega

Desarrollador	Issues Asignados
Guillermo	- [CHORE] Cargar el catalogo de departamentos y municipios de Guatemala - [CHORE] Cargar datos ficticios de demostracion para desarrollo - [CHORE] Migracion: restriccion que impide despachar medicamentos vencidos - [CHORE] Migracion: movimientos de inventario con flujo de aprobacion - [CHORE] Migracion: recetas medicas y su detalle - [CHORE] Migracion: consultas medicas y diagnosticos

Semana 2: PENDIENTE

Desarrollador	Issues Asignados
	- [CHORE] Politicas RLS de pacientes, expedientes y condiciones cronicas - [CHORE] Politicas RLS de atenciones, triaje, consultas y recetas
	- [FEATURE] Validaciones de datos de paciente en shared - [FEATURE] API de pacientes en shared: registrar, consultar y actualizar - [FEATURE] API de busqueda de pacientes por nombre, comunidad y numero de ficha - [FEATURE] Generacion del numero de ficha del expediente
	- [FEATURE] Listado y busqueda de pacientes en web - [FEATURE] Ficha del paciente en web con pestanas - [FEATURE] Formulario de registro de paciente en web - [FEATURE] Edicion de datos del paciente en web
	- [FEATURE] Busqueda de pacientes en la app movil - [FEATURE] Registro de paciente en la app movil replicando la ficha

Desarrollador	Issues Asignados
	física - [FEATURE] Ficha resumida del paciente en la app movil
	- [FEATURE] Validaciones de jornadas y asignaciones en shared - [FEATURE] API de jornadas en shared: registrar, consultar y actualizar - [FEATURE] API de cambio de estado de jornada con transiciones validas - [FEATURE] API de atenciones y cola de pacientes de la jornada
	- [FEATURE] Registro de signos vitales en la app movil - [FEATURE] Registro de la consulta medica en la app movil - [FEATURE] Generacion de la receta medica en la app movil
	- [CHORE] Pruebas de la capa de API compartida - [CHORE] Pruebas de las validaciones de negocio compartidas - [CHORE] Verificar la fidelidad de los formularios con la ficha clinica fisica

Semana 3: PENDIENTE

Desarrollador	Issues Asignados
	- [CHORE] Politicas RLS de medicamentos, lotes, existencias, movimientos y alertas - [CHORE] Politicas RLS de jornadas, personal asignado y proyectos
	- [FEATURE] API de lotes de medicamentos en shared - [FEATURE] API de consulta de existencias en tiempo real - [FEATURE] Validacion de medicamentos vencidos en shared - [FEATURE] API de movimientos de inventario en shared
	- [FEATURE] Historial clinico en linea de tiempo en web - [FEATURE] Consulta de recetas emitidas del paciente en web

Desarrollador	Issues Asignados
	- [FEATURE] Impresion y descarga de la receta medica en web - [FEATURE] API de historial medico completo del paciente
	- [FEATURE] Selecccion de jornada activa en la app movil - [FEATURE] Panel de la jornada en curso en la app movil - [FEATURE] Consulta del historial del paciente en la app movil
	- [FEATURE] Tablero kanban de jornadas en web - [FEATURE] Bandeja de validacion de movimientos para la administradora - [FEATURE] Vista de existencias en tiempo real en web - [FEATURE] Registro de salida de medicamentos en web
	- [FEATURE] Consulta de stock disponible en la app movil - [FEATURE] Registro de entrega de medicamentos en la app movil - [FEATURE] Registro rapido de ingreso de medicamentos en campo
	- [FEATURE] API del reporte de pacientes atendidos en shared - [FEATURE] API del reporte de inventario actual en shared - [FEATURE] Hook de filtros compartidos de reportes

Semana 4: PENDIENTE

Desarrollador	Issues Asignados
	- [CHORE] Politicas RLS de donantes y donaciones - [CHORE] Edge Function programada de alertas de vencimiento
	- [FEATURE] API de registro de donaciones en shared - [FEATURE] Vinculo entre donaciones de medicamentos y el ingreso de inventario - [FEATURE] Exportacion de reportes a CSV en shared

Desarrollador	Issues Asignados
	- [FEATURE] Listado de usuarios en web con filtros - [FEATURE] Formulario de alta de usuario en web - [FEATURE] Pantalla de asignacion de roles y permisos en web
	- [FEATURE] Registro sin conexion con cola de sincronizacion en movil - [FEATURE] Resolucion de conflictos de sincronizacion en movil
	- [FEATURE] Listado y ficha de donantes en web - [FEATURE] Formulario de registro de donacion en web - [FEATURE] Listado y registro de proyectos sociales en web - [FEATURE] Seguimiento del avance de proyectos en web
	- [FEATURE] Dashboard de metricas de impacto en web - [FEATURE] Reporte de pacientes atendidos en web - [FEATURE] Reporte de inventario actual en web - [FEATURE] Resumen de la jornada en la app movil
	- [FEATURE] Panel de indicadores de impacto en web - [FEATURE] Exportacion de reportes a PDF en web - [FEATURE] API de indicadores de impacto en shared

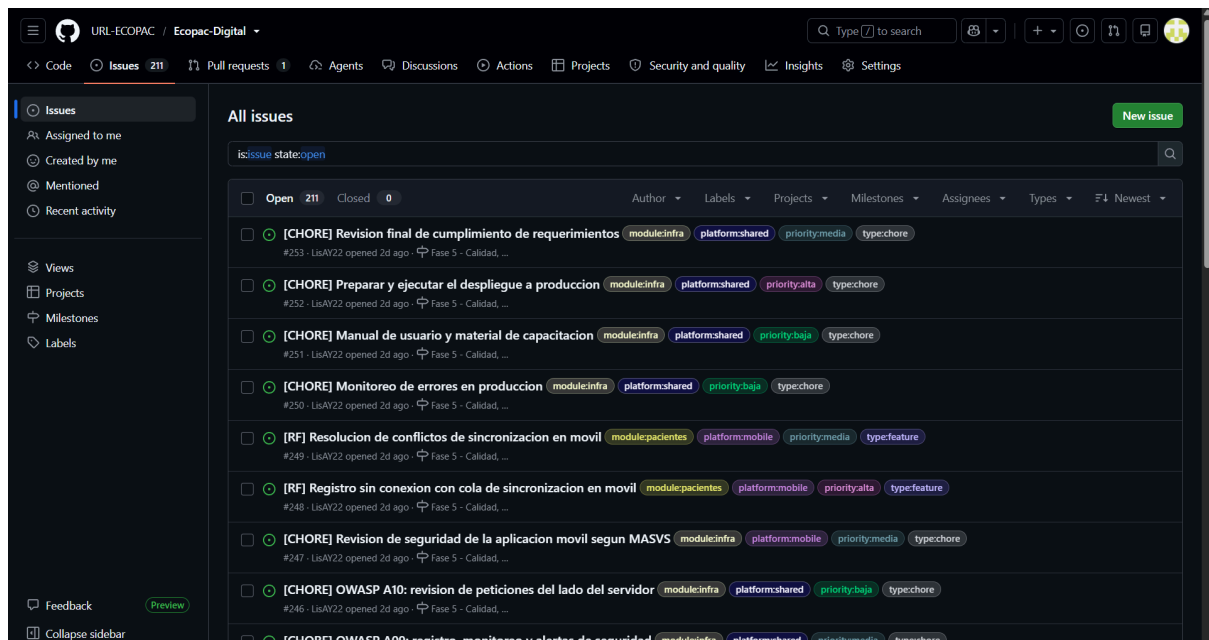
Semana 5: PENDIENTE

Desarrollador	Issues Asignadas
	- [CHORE] Preparar y ejecutar el despliegue a produccion - [CHORE] Configurar y verificar los respaldos de la base de datos - [CHORE] OWASP A05: revision de la configuracion de seguridad - [CHORE] Definir el aprovisionamiento del primer administrador
	- [CHORE] OWASP A01: auditoria del control de acceso - [CHORE] OWASP A03: revision de inyeccion y validacion de

Desarrollador	Issues Asignadas
	entrada - [CHORE] OWASP A10: revision de peticiones del lado del servidor
	- [CHORE] Revision de accesibilidad de la aplicacion web - [CHORE] Verificar el diseno responsivo y la compatibilidad de navegadores - [CHORE] Revision de usabilidad con el personal de la organizacion
	- [CHORE] Revision de seguridad de la aplicacion movil segun MASVS - [CHORE] Medir y optimizar el tiempo de registro en campo - [CHORE] OWASP A02: proteccion de datos sensibles
	- [CHORE] Pruebas end to end de los flujos criticos - [CHORE] Prueba de carga de una jornada de 50 pacientes - [CHORE] Verificar el rendimiento con el crecimiento de la base de datos
	- [FEATURE] Politica de contraseñas y bloqueo por intentos fallidos - [CHORE] OWASP A07: revision de autenticacion y gestion de sesion - [FEATURE] Manejo global de errores y recuperacion ante fallos de red - [CHORE] Monitoreo de errores en produccion
	- [CHORE] Integrar las pruebas en el pipeline de CI - [CHORE] OWASP A06: gestion de dependencias vulnerables - [CHORE] Manual de usuario y material de capacitacion - [CHORE] Revision final de cumplimiento de requerimientos

Producto Backlog

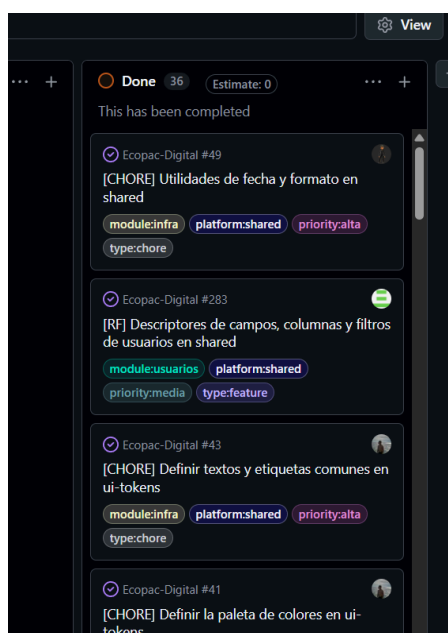
El Product Backlog consolidado representa la totalidad del alcance del proyecto. Actualmente, el 100% de las issues necesarias para terminar el desarrollo de todos los módulos ya han sido identificadas, documentadas y creadas en el repositorio de GitHub. Este listado maestro sirve como la fuente única de verdad de la cual se alimentan las iteraciones semanales.



BACKLOG

Sprint Backlog Inicial

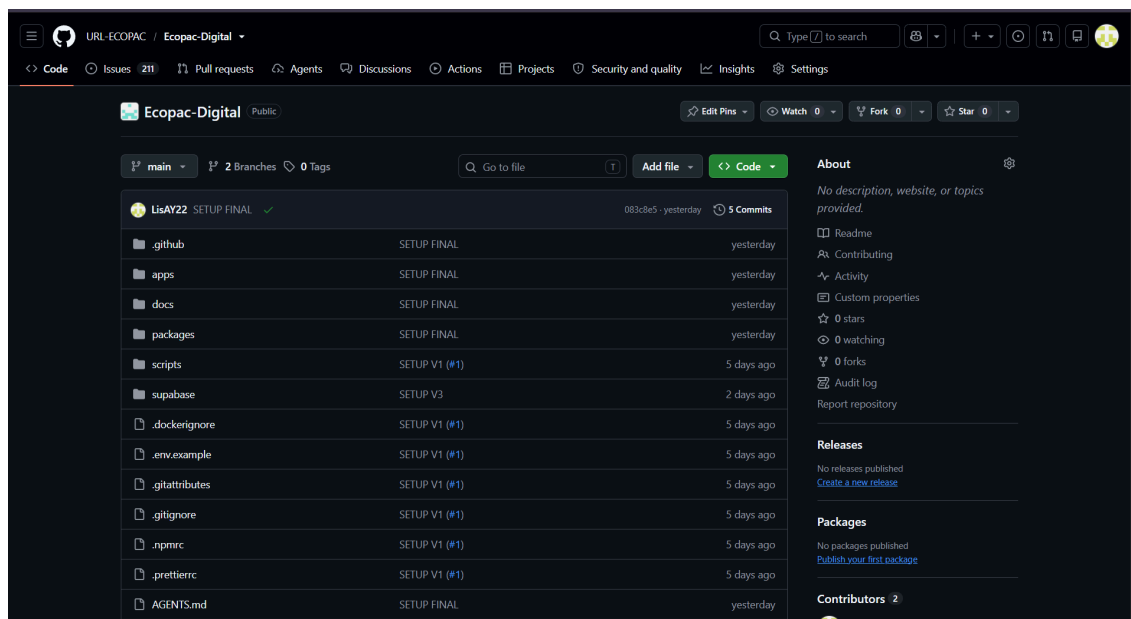
Para la primera semana se asignaron 45 issues distribuidos entre todos los integrantes.



Repositorio GitHub

Se creó y configuró un repositorio completo en GitHub, estableciendo dos ramas protegidas (main y dev) e implementando un flujo de integración y despliegue continuos (CI/CD) para garantizar la calidad del código. Asimismo, se incorporaron los documentos fundamentales de directrices y configuración del proyecto en CONTRIBUTING.md, QUICKSTART.md y README.md. Se ha adjuntado el enlace correspondiente al repositorio para su revisión y seguimiento.

[REPOSITORIO](#)

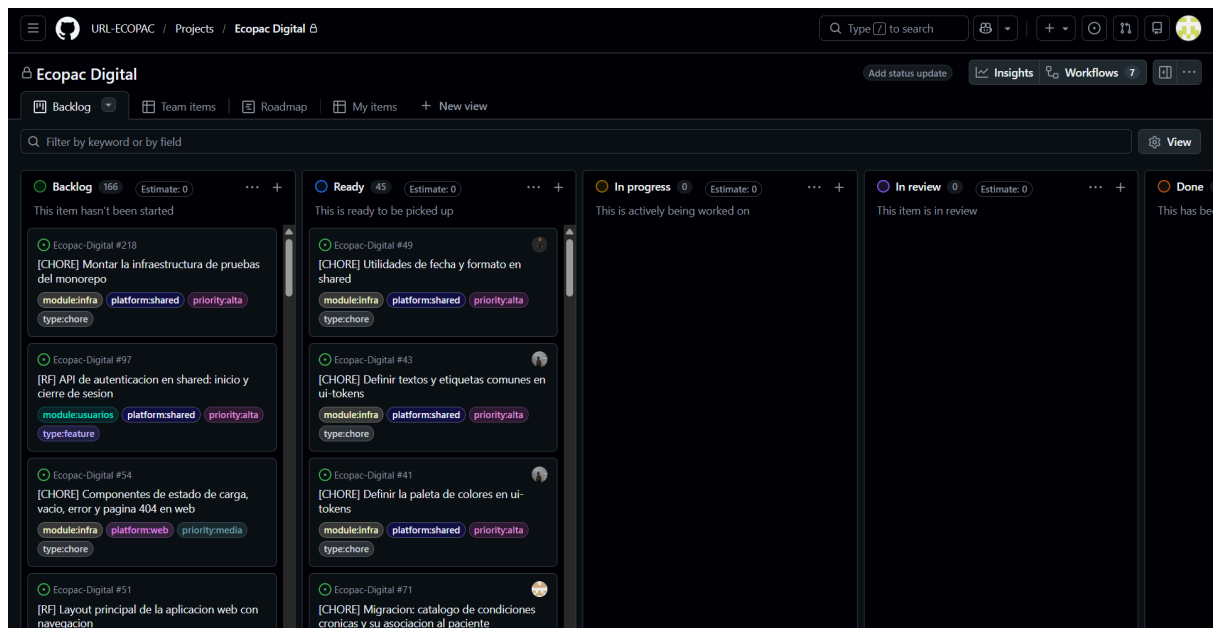


Tablero GitHub Projects

Se configuró un tablero en GitHub Projects para organizar y dar seguimiento a los issues registrados. Este espacio facilita la gestión continua del flujo de trabajo al distribuir los elementos en sus distintos estados: Backlog, Ready, In progress, In review y Done.

Para brindar una visión integral y adaptada a las necesidades del proyecto, el tablero cuenta con diversas vistas estructuradas: la vista de backlog centraliza todos los issues y permite visualizarlas en formato Kanban; la vista de team items agrupa las tareas asignadas a cada integrante; la vista de roadmap proyecta la evolución del trabajo en una línea de tiempo; y la vista my views ofrece un espacio personalizado donde cada miembro puede consultar sus responsabilidades pendientes.

GITHUB PROJECT



Estrategia de Ramas y Control de Versiones

El control de versiones del proyecto se maneja mediante Git, con el repositorio alojado en GitHub bajo la organización URL-ECOPAC. Se adoptó un flujo de trabajo basado en dos ramas principales: main, que contiene el código estable listo para producción, y develop, que funciona como rama de integración y staging donde convergen los avances del equipo antes de ser promovidos a producción. Cada nueva funcionalidad o corrección se desarrolla en una rama individual derivada de develop, siguiendo la convención de nombres `feature/<módulo>-<descripción>` o `fix/<descripción>`, lo cual permite trazar cada rama directamente al requerimiento funcional o issue que la origina.

La integración de cambios se realiza exclusivamente mediante Pull Requests hacia develop, los cuales requieren la revisión aprobada de la líder de grupo y que los verificadores automáticos de integración continua pasen exitosamente antes de poder fusionarse; ambas ramas principales cuentan con reglas de protección que impiden el push directo. La promoción de develop hacia main está reservada al Project Manager del equipo, y se realiza una vez que el conjunto de funcionalidades correspondiente a un entregable ha sido validado. Este esquema busca mantener la estabilidad de la rama de producción mientras permite un desarrollo ágil y paralelo entre los ocho integrantes del equipo.

El detalle completo de esta estrategia se encuentra documentado en:

[CONTRIBUTING.md](#)

Definición de ambientes: desarrollo, pruebas y producción.

El ciclo de vida del software se gestiona a través de tres entornos principales, diseñados para separar el desarrollo, la validación y la operación final:

Ambiente de Desarrollo

Es el entorno inicial donde se realiza la construcción de nuevas funcionalidades, experimentación y corrección de errores. Se ejecuta localmente en la máquina de cada desarrollador. La aplicación web se levanta preferiblemente a través de contenedores Docker, mientras que la aplicación móvil utiliza el servidor local de Expo. Los desarrolladores utilizan el stack local de Supabase (supabase start) para probar cambios y migraciones en el esquema de la base de datos de manera segura, o bien se conectan temporalmente a la base de datos de desarrollo mediante las llaves públicas configuradas en el archivo `.env.development`. Se utilizan ramas de trabajo efímeras creadas para cada ticket (feature/*, fix/*, chore/.*).

Ambiente de Pruebas

Es el entorno de consolidación. Sirve para validar que la integración del código de distintos desarrolladores funcione correctamente y cumpla con los estándares de calidad antes de su lanzamiento. La base de datos y la autenticación apuntan al proyecto en la nube Supabase.

Estrategia de Pruebas y Validación

Como regla general del equipo, en cada issue se exige agregar pruebas si es necesario, garantizando que siempre exista cobertura para validar los nuevos cambios aportados. Se cuenta con workflows automatizados de CI/CD (GitHub Actions) que se disparan en cada Pull Request. Estos flujos revisan el código (npm run lint), verifican la compilación (npm run build) y levantan bases de datos locales desechables para asegurar que ninguna integración rompa el sistema. Adicionalmente, se planifican issues específicas al final de cada ciclo de desarrollo dedicadas exclusivamente a realizar pruebas finales integrales (QA) sobre todo el sistema.

Ambiente de Producción

Es el entorno real, estable y definitivo que utilizan los usuarios finales durante la planificación y ejecución de las jornadas médicas. La aplicación web se sirve desde la configuración de producción principal en Vercel, y todo el backend se conecta al proyecto definitivo Supabase. Los secrets y contraseñas de producción están estrictamente gestionados en la plataforma de CI/CD y nunca se exponen ni se suben al repositorio.

Normas de Codificación

- **Arquitectura de Código:** El proyecto sigue una estricta regla de separación de lógica. Todo el código que no sea visual (reglas de negocio, llamadas a la base de datos, validaciones y tipos) debe residir en `packages/shared`. Las aplicaciones web y mobile únicamente importan esta lógica para envolverla en componentes de interfaz (JSX/CSS), evitando así la duplicación de código entre plataformas.
- **Control de Versiones y Ramas:** Se utiliza un Git Flow simplificado con `main` para producción y `develop` para integración. Todo desarrollo debe hacerse en ramas creadas a partir de `develop`, usando la nomenclatura `feature/*`, `fix/*` o `chore/*`. Cada rama debe corresponder a un único issue y eliminarse tras ser fusionada.
- **Commits y Pull Requests (PR):** Es obligatorio utilizar Conventional Commits. Para aprobar un PR hacia `develop`, el código debe pasar las validaciones de CI, referenciar el ticket con Closes #X y fusionarse exclusivamente mediante Squash and merge.
- **Base de Datos y Migraciones:** El esquema de Supabase no se edita manualmente en producción. Todos los cambios en la estructura de datos se manejan mediante archivos `.sql` en `supabase/migrations/`, los cuales se prueban de forma local antes de llegar a los entornos principales.
- **Seguridad y Accesos:** Está prohibido subir llaves reales al repositorio; solo se mantiene el archivo `.env.example`. Además, los permisos de acceso a los datos deben estar protegidos por políticas RLS en la base de datos, no solo en el frontend.
- **Manejo de Dependencias:** Se emplean versiones fijas para dependencias críticas como React Native y se maneja una actualización controlada en el entorno móvil (Expo). Los paquetes compartidos deben mantenerse sin dependencias exclusivas de plataformas móviles.

El detalle completo de las normas se encuentra en:

[DOCS](#)

Matriz de Riesgos

ID	Categoría	Descripción del Riesgo	Impacto	Probabilidad	Nivel	Estrategia de Mitigación
R01	Seguridad	Acceso no autorizado o fuga de datos clínicos sensibles de pacientes.	Alto	Media	Alto	Políticas de seguridad a nivel de fila (RLS) en Supabase y cifrado en tránsito (TLS/HTTPS).
R02	Técnico	Fallo o corrupción durante las migraciones de base de datos y catálogos geográficos.	Alto	Media	Alto	Scripts de migración versionados, rollback automático y pruebas de integridad relacional previa.
R03	Desarrollo	Inconsistencias entre los entornos de desarrollo,	Medio	Media	Medio	Contenedorización del entorno con Docker y

		pruebas y producción.				variables de entorno centralizadas.
R04	Datos	Inconsistencia lógica en el registro de formularios de atención médica.	Medio	Baja	Bajo	Validaciones en frontend/backend e implementación de triggers e imperturbabilidad de datos en base de datos.

Plan de Acción y Monitoreo

- **Controles Preventivos:** Auditar periódicamente los scripts de migración de Supabase antes de aplicarlos en entornos compartidos y validar las políticas de RLS.
- **Controles Correctivos:** Si ocurre una desconexión en campo, las aplicaciones locales deben conmutar automáticamente a modo IndexedDB sin interrumpir la captura de datos del paciente.
- **Monitoreo Continuo:** Revisión periódica de la matriz al finalizar cada iteración de desarrollo o cambio de arquitectura en el repositorio.

Registro de Problemas y Cambios

Para formalizar el histórico de desviaciones técnicas y decisiones arquitectónicas de Ecopac Digital, este apartado integra el control de cambios crítico, destacando la transición de arquitectura, y el protocolo de resolución de incidentes durante el ciclo de vida del desarrollo.

Registro de Cambios Arquitectónicos

ID Cambio	Fecha	Componente	Descripción del Cambio	Justificación Técnica / Impacto	Estado
CHG-001	Agosto 2026	Capa de Backend y BD	Migración de Django a Supabase	Costo y Mantenimiento Se eliminó la necesidad de desplegar y mantener un servidor Django en PaaS de pago o contenedores con limitaciones de <i>cold-start</i>. Ventajas Integración directa de RLS en PostgreSQL, API REST/GraphQL generada automáticamente , Auth integrado y soporte nativo para Storage, manteniendo el presupuesto.	Aprobado y se está implementando

CHG-002	Agosto 2026	Estrategia Auth & RLS	Reemplazo de middleware de autorización custom (Django ORM) por Row Level Security (RLS) nativo de PostgreSQL .	Traslada la responsabilidad de la seguridad del nivel de aplicación al motor de base de datos. Garantiza que la service_role key permanezca aislada y que clientes web/móviles interactúen mediante la anon key restringida por JWT.	Aprobado y se esta implementando
----------------	-------------	-----------------------	---	--	----------------------------------

Registro de Problemas Conocidos y Mitigaciones

Durante las fases de diseño y pruebas preliminares de la nueva arquitectura sobre Supabase, se identificaron los siguientes problemas operativos y sus soluciones:

ISSUE-01: Exposición Potencial de Datos por Tablas sin RLS Habilitado

- Problema: Al migrar desde Django, las nuevas tablas creadas en PostgreSQL no aplican denegación por defecto si no se habilita explícitamente RLS.
- Impacto: Riesgo crítico de fuga de expedientes clínicos si se consulta la API REST.
- Mitigación: Se incluyó una verificación en el pipeline CI/CD y en la checklist de salida que valida que el 100% de las tablas en el esquema public posean RLS activo antes de autorizar el despliegue.

ISSUE-02: Pausa Automática del Proyecto Supabase por Inactividad

- Problema: El entorno gratuito congela la base de datos tras 7 días sin tráfico, provocando fallos en la app móvil y web.
- Impacto: El sistema puede cargar lento si no se usa por varios días consecutivos.
- Mitigación: Implementación de un workflow en GitHub Actions que ejecuta consultas de mantenimiento cada 3 días, garantizando actividad continua y cero tiempo de inactividad por congelamiento.

ISSUE-03: Inconsistencias por Descuento Directo de Inventario desde el Cliente

- Problema: Si el cliente móvil descontaba stock directamente al emitir una receta, fallas de red generaban registros duplicados o existencias negativas.
- Impacto: Crítico en el control de lotes y medicamentos.
- Mitigación: Creación del flujo en dos pasos: los movimientos quedan en estado pendiente y solo un trigger/función ejecutada por un rol autorizado aplica la actualización atómica del stock mediante la restricción.

Procedimiento General para la Gestión de Nuevos Cambios

1. Documentación del Cambio: Cualquier modificación en la estructura de la base de datos o en la arquitectura debe registrarse en el *Architectural Change Log* indicando justificación e impacto.
2. Validación de Seguridad: Todo cambio en la capa de datos requiere la creación/actualización de migraciones de código SQL y sus correspondientes políticas RLS.
3. Respaldo Pre-Migración: Antes de aplicar cualquier script de migración en el entorno de Staging o Producción, se ejecutará un respaldo inmediato de la base de datos.
4. Ejecución de Pruebas de Regresión: La migración se considerará completada únicamente tras aprobar el 100% de los casos de prueba marcados como críticos y altos.

Plan Inicial de Pruebas

Alcance

- Aplicación web administrativa (React + Vite + Bootstrap, alojada en Vercel).
- Aplicación móvil de campo (React Native + Expo).
- Capa de servicios y base de datos (Supabase / PostgreSQL: API REST, Auth, Storage, RLS, Edge Functions/rutinas programadas).
- Reglas de negocio críticas: validación de movimientos de inventario, alertas de vencimiento, flujo clínico, control de roles y permisos.
- Seguridad: autenticación, RLS, cifrado en tránsito, exposición de llaves (anon vs service_role).
- Comportamiento operativo de la infraestructura gratuita (reposito por inactividad de Supabase, límites de Vercel/Expo).

Estrategia de pruebas

Niveles de prueba

Nivel	Qué cubre	Responsable	Herramientas
Unitarias	Funciones puras, hooks, utilidades, validaciones de formularios, funciones PL/pgSQL	Desarrollador que escribe el código	Vitest/Jest (React y RN), pgTAP o tests SQL manuales para funciones/triggers
Integración	Interacción componente-Supabase (API REST, Auth, Storage), triggers y RLS en conjunto	Desarrollador + QA interno	Supertest/fetch mocks, Postman/Insomnia contra el proyecto Supabase de test

Sistema (funcional end-to-end)	Flujos completos: registro de paciente → jornada → triaje → consulta → receta → descuento de stock	QA interno / equipo rotativo	Pruebas manuales guiadas por casos de prueba; Playwright/Cypress para regresión web si el tiempo lo permite
Aceptación (UAT)	Validación con la organización (Ecopac) sobre escenarios reales de jornada	Representante de Ecopac + equipo	Sesión guiada con checklist, ambiente de staging

Tipos de prueba

- Funcional: cada historia de usuario del Product Backlog debe tener al menos un caso de prueba de aceptación.
- De seguridad y control de acceso: verificar que las políticas RLS bloquean correctamente el acceso cruzado entre roles (Administrador, Junta Directiva, Médico, Voluntario), y que la service_role key nunca queda expuesta en el bundle cliente.
- De integridad de datos: validar que las reglas críticas (movimiento "pendiente" hasta aprobación, chk_existencia_no_negativa, unicidad de lote, triggers de existencias) se sostienen incluso si el cliente intenta saltárselas.
- De usabilidad: validar wireframes/prototipo navegable con usuarios reales (personal médico/voluntario, muchos con poca alfabetización digital).
- De compatibilidad: dispositivos Android de gama media/baja, distintos tamaños de pantalla web (según Bootstrap breakpoints).
- De infraestructura / operación: comportamiento ante el reposo de Supabase a los 7 días, tiempo de reactivación, funcionamiento del cron de GitHub Actions que evita la pausa, y de los respaldos automáticos.
- De rendimiento básico: tiempos de carga de dashboard, listado de pacientes/inventario con volumen simulado (ej. 2,000 pacientes, 500 lotes) para verificar que se mantiene dentro de límites razonables en el plan gratuito.

Casos de prueba iniciales por módulo

Esta es la primera capa de casos, priorizados por riesgo. Se ampliarán en un documento de casos de prueba detallado por sprint.

Autenticación y permisos (RLS)

ID	Caso	Resultado esperado	Prioridad
AUTH-01	Login con credenciales válidas por cada rol	Acceso concedido, sesión JWT emitida	Alta
AUTH-02	Bloqueo tras intentos fallidos repetidos	Cuenta bloqueada temporalmente	Alta
AUTH-03	Voluntario intenta leer tabla consulta (diagnóstico) vía API directa	RLS deniega el acceso, no solo la UI	Crítica
AUTH-04	Médico intenta ver expedientes fuera de una jornada activa	RLS deniega el acceso	Crítica
AUTH-05	Verificar que service_role key no aparece en el bundle de producción (web ni APK)	No se encuentra en el código compilado	Crítica
AUTH-06	Expiración de sesión por inactividad en dispositivo móvil compartido	Sesión se cierra y exige nuevo login	Alta

Pacientes y expediente clínico

ID	Caso	Resultado esperado	Prioridad
PAC-01	Registrar paciente sin DPI, solo con nombre y comunidad	Se crea correctamente, búsqueda posterior funciona por nombre/comunidad	Alta
PAC-02	Buscar paciente con nombre similar (typo)	El sistema sugiere coincidencias por similitud	Media

PAC-03	Paciente menor de edad sin fecha de nacimiento exacta	Campo es_menor_edad se calcula correctamente con edad aproximada	Media
PAC-04	Intento de eliminar paciente con consultas médicas asociadas	El trigger de soft-delete lo impide	Alta

Jornadas y atención clínica

ID	Caso	Resultado esperado	Prioridad
JOR-01	Flujo completo: registro → triaje → consulta → receta → descuento de stock	Cada paso queda registrado y el stock se descuenta solo tras aprobación	Crítica
JOR-02	Registrar dos atenciones del mismo paciente en la misma jornada	Restricción uq_paciente_jornada lo impide o lo maneja según regla de negocio	Media
JOR-03	Cambiar estado de jornada en el tablero Kanban	Estado se actualiza y queda historial en jornada_estado_historial	Alta
JOR-04	Triaje con presión sistólica menor que la diastólica	Constraint chk_presion rechaza el registro	Media

Inventario, lotes y movimientos

ID	Caso	Resultado esperado	Prioridad
INV-01	Registrar salida de medicamento por dispensación	Movimiento queda en estado "pendiente de validación"	Crítica
INV-02	Administrador aprueba movimiento pendiente	Existencia se actualiza correctamente vía trigger	Crítica
INV-03	Intento de salida mayor a la existencia disponible	El sistema rechaza o genera inconsistencia negativa (debe rechazar)	Crítica

INV-04	Traslado entre bodegas	Se descuenta en origen y se suma en destino de forma atómica	Alta
INV-05	Lote próximo a vencer (30 días)	Se genera alerta automática	Alta
INV-06	Rechazo de movimiento sin motivo	Constraint chk_rechazo_con_motivo lo impide	Media
INV-07	Voluntario consulta stock fuera del botiquín de su jornada activa	RLS restringe la vista solo a su jornada	Alta

Reportes

ID	Caso	Resultado esperado	Prioridad
REP-01	Junta Directiva accede a vista_reporte_impacto	Solo lectura, sin acceso a datos crudos de pacientes	Alta
REP-02	Reporte con datos de varias jornadas y donaciones	Los totales agregados coinciden con el detalle fuente	Alta

Infraestructura / operación

ID	Caso	Resultado esperado	Prioridad
OPS-01	Proyecto Supabase tras 7 días sin actividad simulada	Entra en reposo; el cron de GitHub Actions lo evita en condiciones normales	Alta
OPS-02	Reactivación manual del proyecto	Disponible en ~30 segundos, sin pérdida de datos	Alta
OPS-03	Verificar que el respaldo automático programado se ejecutó	Archivo de respaldo reciente existe en el repositorio/almacenamiento designado	Crítica

OPS-04	Instalación del APK generado por Expo en un dispositivo Android real de campo	Instalación exitosa tras habilitar "orígenes desconocidos"	Media
--------	---	--	-------

Gestión de defectos

Se clasifican por severidad:

Severidad	Definición	Ejemplo
Crítica	Pérdida de datos, bypass de seguridad/RLS, inconsistencia de stock	Un voluntario puede leer diagnósticos
Alta	Rompe un flujo principal sin workaround	No se puede aprobar un movimiento de inventario
Media	Afecta la experiencia pero tiene alternativa	Búsqueda de paciente lenta pero funcional
Baja	Cosmético	Alineación de un botón

Los defectos se registran en el tablero de gestión definido en la sección de planificación, con etiqueta de módulo y severidad, y se priorizan antes de cada sprint siguiente.

Plan de Respaldo y Recuperación

El plan de respaldo y recuperación de datos DRP para Ecopac Digital establece los procedimientos, responsabilidades y tecnologías necesarias para garantizar la continuidad operativa ante cualquier eventualidad que comprometa la integridad o disponibilidad de los datos del sistema.

El plan se estructura para cumplir con los criterios más importantes del proyecto: costo cero, simplicidad operativa y madurez técnica. Además de tener limitaciones en la arquitectura como la plataforma de la base de datos y otras herramientas utilizadas.

Objetivos

Garantizar la continuidad operativa de las jornadas médicas y la integridad de los expedientes clínicos electrónicos y el control de inventario, definiendo los lineamientos para la protección, almacenamiento, verificación y restauración de la información del sistema Ecopac Digital ante incidentes, errores humanos o fallas en la infraestructura de la nube.

Alcance e inventario de activos

El plan cubre todos los componentes de la arquitectura de Ecopac Digital, los cuales operan bajo servicios en la nube de nivel gratuito.

Sistema o Servicio	Nombre / Proveedor	Datos Protegidos	Criticidad	Dependencias
Base de Datos	Supabase (PostgreSQL)	Tablas: Expedientes clínicos, lotes, movimientos, usuarios.	Crítico	Autenticación, GitHub Actions.
Almacenamiento	Supabase Storage	Fotografías de fichas físicas, comprobantes de donación.	Alto	Base de datos
Código Fuente	Repositorio GitHub	Código React (Web) y React Native (Móvil).	Alto	N/A
Hosting Web	Vercel (Plan Hobby)	Interfaz administrativa.	Medio	Repositorio GitHub, Supabase API.

Compilación Móvil	Expo Cloud	Archivos instalables (APK).	Medio	Código fuente, Supabase API.
Documentación		Manuales, políticas, diagramas y procedimientos	Medio	Repositorio institucional

Nota importante: Los respaldos nativos de Supabase solo están disponibles en planes de pago, no incluyen los archivos almacenados en Storage.

Objetivos de recuperación

Dada la naturaleza de la organización y el uso de infraestructura de costo cero, se establecen métricas que equilibran la criticidad clínica con las capacidades técnicas.

Sistema / Componente	RPO: Pérdida Máxima Aceptable	RTO: Tiempo de Recuperación	Prioridad
Base de Datos Clínica e Inventario	24 horas, mediante el último volcado diario	4 horas	1
Archivos Adjuntos/ Storage	7 días, mediante la última copia semanal	24 horas	2
Aplicación Web	N/A Código en GitHub	2 horas	3
Aplicación Móvil	N/A Código en GitHub	12 horas	4
Retención de respaldos	30 días	N/A	5

Estrategia de respaldos

Al carecer de respaldos automáticos nativos en el plan de Supabase, se implementa la siguiente arquitectura aplicando la regla 3-2-1

Copia 1	Datos originales alojados en los servidores de Supabase.
Copia 2 Nube Externa - Automatizada	Ejecución de rutinas mediante GitHub Actions para extraer volcados SQL de PostgreSQL. Estos se enviarán automáticamente a un almacenamiento en la nube institucional de la directiva como Google Drive o OneDrive de la ONG.
Copia 3 offline o fisica	Descarga mensual de los volcados SQL y respaldos de Storage a un disco duro externo custodiado físicamente por Ecopac.

Respaldo de la base de datos

Herramienta: Supabase CLI + GitHub Actions

Frecuencia:

- Respaldo diario completo de esquema y datos.
- Respaldo adicional antes de cada cambio estructural.
- Respaldo inmediato antes de ejecutar migraciones.
- Respaldo mensual de largo plazo.

Mecanismo: El flujo de respaldo automatizado se implementa mediante un workflow de GitHub Actions.

- Se conecta a la base de datos de Supabase utilizando la cadena de conexión del Session Pooler.

- Ejecuta el comando `supabase db dump` para generar tres archivos de respaldo separados:
 - `roles.sql` — definiciones de roles y permisos.
 - `schema.sql` — estructura completa de la base de datos (tablas, vistas, índices, funciones, triggers)
 - `data.sql` — datos de todas las tablas
- Excluye tablas del esquema `storage` que no son necesarias para la restauración lógica
- Automatiza el commit de los archivos generados al repositorio privado de respaldos

Almacenamiento de respaldos

Los archivos de respaldo se almacenan en un repositorio privado de GitHub dedicado exclusivamente a este fin. Este repositorio debe ser:

- Privado
- Accesible solo por el equipo de desarrollo y la administración de Ecopac
- Versionado, permitiendo recuperar cualquier versión histórica

Cifrado de respaldos

Para proteger la información sensible, se recomienda añadir un paso de cifrado al workflow:

- Utilizar `gpg` o `openssl` para cifrar los archivos `.sql` antes del commit
- Almacenar la clave de cifrado como secreto en GitHub Actions
- Documentar el procedimiento de descifrado para la recuperación

Respaldo de archivos

Los respaldos nativos de Supabase no incluyen los archivos de Storage.

Mecanismos propuestos

Opción	Descripción	Costo
Opción A: Sincronización periódica	Script que descarga todos los archivos de Storage mediante la API de Supabase y los sube a un almacenamiento alternativo (GitHub Releases, Cloudflare R2, o Google Drive)	Cero
Opción B: Respaldo manual	Descarga manual de los archivos desde el dashboard de Supabase cada cierto período	Cero

Opción C: GitHub LFS	Almacenar los archivos en el mismo repositorio de respaldos usando Git LFS	Gratuito hasta 1 GB
-----------------------------	--	---------------------

Respaldo y configuración de variables de entorno

Elemento	Método de respaldo	Frecuencia
Variables de entorno (Vercel)	Documentar en un archivo seguro (Bitwarden / 1Password)	A cada cambio
Claves de API de Supabase	Respaldar en GitHub Secrets y documentar	A cada cambio
Políticas RLS	Incluidas en schema.sql	Diario
Configuración de Supabase	Documentar en el repositorio del proyecto	A cada cambio

Procedimientos de recuperación y escenarios posibles

Escenario 1: Pérdida o corrupción de datos en la base de datos

Síntomas: Tablas con datos incorrectos, eliminación accidental de registros, corrupción de datos.

Procedimiento:

- Identificar el punto de restauración: Seleccionar el archivo de respaldo más reciente anterior al incidente desde el repositorio de respaldos.
- Crear un nuevo proyecto en Supabase.
- Restaurar roles y esquema
- Restaurar datos
- Reconfigurar extensiones y webhooks si eran utilizadas en el proyecto original.
- Actualizar la cadena de conexión en las aplicaciones web y móvil apuntando al nuevo proyecto.

- Verificar la integridad de los datos restaurados.

Escenario 2: Proyecto pausado por inactividad

Contexto: El plan gratuito de Supabase pausa los proyectos tras 7 días de inactividad.

Síntomas: La aplicación no puede conectarse a la base de datos.

Procedimiento:

- Acceder al dashboard de Supabase.
- Identificar el proyecto pausado en estado "Paused".
- Hacer clic en "Restore project", teniendo en cuenta que la reactivación tarda aproximadamente 30 segundos.
- Verificar que el proyecto muestre estado "Active" antes de ejecutar consultas.
- Confirmar que los datos están intactos realizando una consulta de prueba.

Prevención: El workflow de respaldo diario que ejecuta consultas a la base de datos mantiene el proyecto activo automáticamente, evitando la pausa por inactividad.

Escenario 3: Proyecto eliminado permanentemente

Contexto: Si un proyecto en plan gratuito permanece pausado más allá del período de gracia, Supabase lo elimina permanentemente.

Síntomas: El proyecto ya no aparece en el dashboard o aparece como "Deleted".

Procedimiento:

- Crear un nuevo proyecto en Supabase.
- Restaurar desde el respaldo más reciente siguiendo el procedimiento del Escenario 1.
- Actualizar todas las cadenas de conexión en Vercel y en la aplicación móvil.
- Reconfigurar las políticas de autenticación en el nuevo proyecto.
- Verificar la operación completa del sistema.

Prevención: Configurar el workflow de respaldo con una alerta que notifique si el proyecto no puede ser accedido durante varios días consecutivos.

Roles y responsabilidades

Rol	Responsabilidad
Administrador del sistema	Supervisar la ejecución de respaldos; verificar periódicamente que las copias se están generando correctamente; ejecutar la recuperación en caso de incidente.
Equipo de desarrollo	Configurar y mantener el workflow de respaldos; documentar procedimientos; probar la restauración periódicamente.
Personal de campo	Reportar cualquier anomalía en el acceso a los datos durante las jornadas.

Pruebas y mantenimiento

- Revisión Quincenal: La Presidenta o un delegado revisará visualmente la carpeta de destino en la nube para confirmar que los archivos SQL se estén creando con el tamaño y fecha correctos.Simulacro
- Revisión Semestral: El equipo de TI creará un proyecto paralelo temporal en Supabase e inyectará un respaldo de prueba para medir el tiempo real de restauración (RTO) y validar que el archivo SQL no esté corrupto.
- Prevención de Pausa (Continua): Supervisar que la rutina automatizada en GitHub Actions se ejecute cada 3 días para evitar que Supabase congele el proyecto por inactividad.

Tiempo de retención de respaldos

Tipo de copia	Retención
Copias diarias	30 días
Copias semanales	12 semanas
Copias mensuales	12 meses

Copia anual	Según política institucional
Copias previas a migraciones	Hasta validar la migración
Copias de incidentes	Hasta cerrar la investigación

Validaciones posteriores a la restauración

Antes de autorizar el uso normal, comprobar:

Base de datos

- Las tablas existen.
- Las relaciones funcionan.
- Las claves foráneas están activas.
- Los triggers funcionan.
- Las funciones se ejecutan correctamente.
- Las restricciones de inventario siguen activas.
- No existen existencias negativas.
- Los movimientos aprobados modifican correctamente el inventario.

Seguridad

- RLS está habilitado en todas las tablas sensibles.
- Un voluntario no puede consultar diagnósticos.
- Un médico solo puede acceder a la información autorizada.
- La junta directiva no puede modificar datos clínicos.
- Solo la administradora puede aprobar movimientos.
- Los buckets de Storage no son públicos.
- La service_role key no está en los clientes.
- Los usuarios inactivos no pueden iniciar sesión.

Aplicaciones

- La aplicación web inicia sesión.
- La aplicación móvil se conecta.
- Se pueden registrar pacientes.

- Se puede consultar un expediente autorizado.
- Se puede crear una consulta médica.
- Se puede registrar un movimiento pendiente.
- Se puede aprobar un movimiento.
- Se actualiza la existencia.
- Se pueden consultar los reportes.
- Se conservan los eventos de auditoría.